

Parte 1:

1 y 2. control_points.py

3. hsv_vs_lch.py

Las regiones afectadas son las mismas, pero difieren notoriamente: en HSV los colores se vuelven más brillantes, mientras que en LCH se ven cambios más densos y naturales. Esto se debe a cómo cada modelo maneja la luz. El espacio LCH imita la visión humana separando la luz del color por completo. Al aumentar el croma, la luminosidad se mantiene estrictamente intacta, mientras que al aumentar la saturación esta aumenta el brillo e intensidad.

PD: Se tuvieron que reescalar los puntos para el modelo LCH porque si no lch (sin los puntos de control reescalados) saturaba toda la imagen.

4. Una amplitud extrema de m empuja los colores fuera de rango generando clipping que aplanan la textura de la imagen. Respecto a los puntos de control, si están muy juntos o son demasiados, la interpolación lineal crea pendientes abruptas que rompen los degradados naturales produciendo banding, mientras que usar muy pocos puntos altera colores vecinos no deseados. Finalmente, la decisión de diseñar g_m como un factor multiplicativo es útil porque protege a los tonos grises de adquirir tintes artificiales.

5. caso_clipping.py

Al tener un m y usar el modo LHC, mi implementación se genera una pérdida de detalle cromático y hace que la imagen se vea más borrosa.

Parte 2:

1. ecualización_clasica.py

2. overlap_densidad.py

Se observa que a mayor overlap mejor se disimulan los cambios bruscos entre regiones. Esto ocurre porque el valor de cada píxel resulta del promedio interpolado de múltiples ventanas vecinas, eliminando el efecto de bloque. Por otro lado, al aumentar la densidad de la malla (usando ventanas muy pequeñas), se genera un incremento significativo de ruido visible.

3. bins.py

Zonas 128: Si se quitan algunos tonos, casi no se nota. Se tiene suficiente información visual para que la imagen se siga viendo con detalle. Pero, este detalle se pierde cuando se usan pocos bins.

zonas 32: Con 256 tonos, la imagen se llena de un "granulado" muy molesto. Sin embargo, al bajar a 64 tonos, ese granulado desaparece mágicamente; los colores se ven obligados a agruparse y la imagen se ve más limpia.

4. restricción.py

5. comparación.py

Mientras la ecualización global oscurece ciertas áreas, los métodos locales logran extraer eficientemente el contraste de las texturas más pequeñas. Sin embargo, la versión local sin límite fracasa al amplificar severamente el ruido en zonas homogéneas, destruyendo la suavidad del agua o el cielo. Por su parte, tanto la "Local Limitada" como "CLAHE" solucionan este problema protegiendo las áreas lisas.

6. res_indeseado.py

Ambos métodos solucionan el problema poniendo un clip_limit. Logrando así un equilibrio. Resaltan los detalles, es decir, hacen que las texturas finas, como cada pelo y bigote del tigre, se vean mucho más nítidos y definidos. Asimismo, protegen el fondo, evitando así que aparezca ese molesto "granulado" en las zonas oscuras y desenfocadas.

7. frontera.py

El mecanismo de solapamiento funciona como un difuminador que sirve para borrar los cortes cuadrados que se forman por las regiones. Al superponer las ventanas de análisis, los bordes de los bloques no chocan de forma brusca, sino que se entrelazan.

Parte 3:

1,2,3,4. cuatro_factores.py

Al realizar reducciones, el método del vecino más cercano genera severos problemas estructurales y pérdida de simetría, como se observa en la irregularidad de los bloques del tablero, mientras que la interpolación bilineal conserva mejor la geometría a cambio de introducir un ligero desenfoco en los detalles y texturas.

En las ampliaciones el vecino más cercano produce un fuerte efecto de pixelado y bordes escalonados al duplicar píxeles adyacentes. Por el contrario, la interpolación

bilineal suprime el pixelado creando transiciones suaves mediante el promedio de valores, lo que resulta en una imagen con bordes continuos, pero visiblemente más borrosa.

5. regreso.py

Al aplicar la reducción, la cantidad de píxeles disminuye y se descarta la información de los bordes, creando franjas grises intermedias donde antes había un corte abrupto. Cuando se aplica la ampliación para volver a la resolución inicial, la imagen recuperada muestra bordes borrosos y desenfocados en comparación con la versión original.

6. reescalado_sucesivo.py

7. aliasing.py

El origen del aliasing es un problema de submuestreo espacial. Al encoger la imagen, la nueva cuadrícula simplemente no tiene los píxeles suficientes para capturar la alta frecuencia de los cambios rápidos entre los cuadros blancos y negros originales